DATASCIENTEST · DEVOPS PROJECT · PRE-DEFENSE

Production-ready delivery of a FastAPI app

Jenkins + Argo CD GitOps on k3s

Vitalii TYTSKYI · Dionis FRANGU · Tatiana ZAINULLINA

Team work: every approach was discussed and decided together

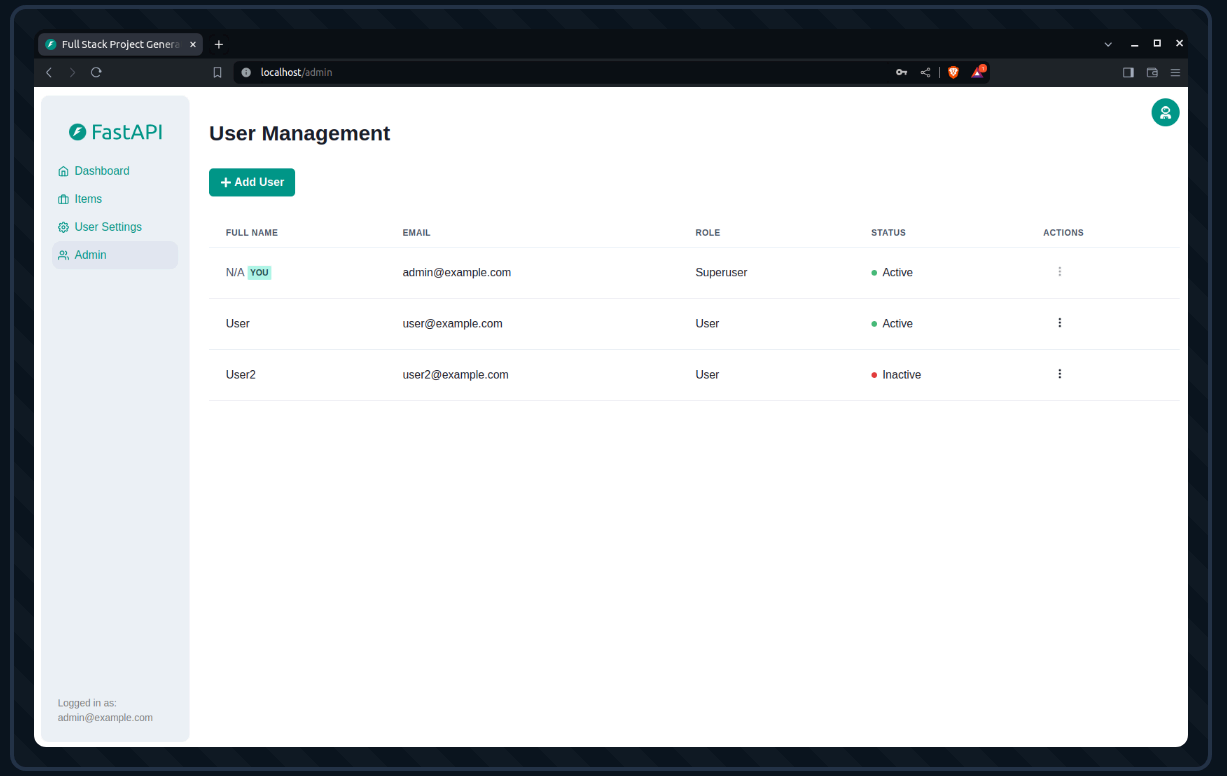
01 Project goal & application

> **Goal:** build, test, deploy, secure, monitor and recover an application the way a real engineering team does.

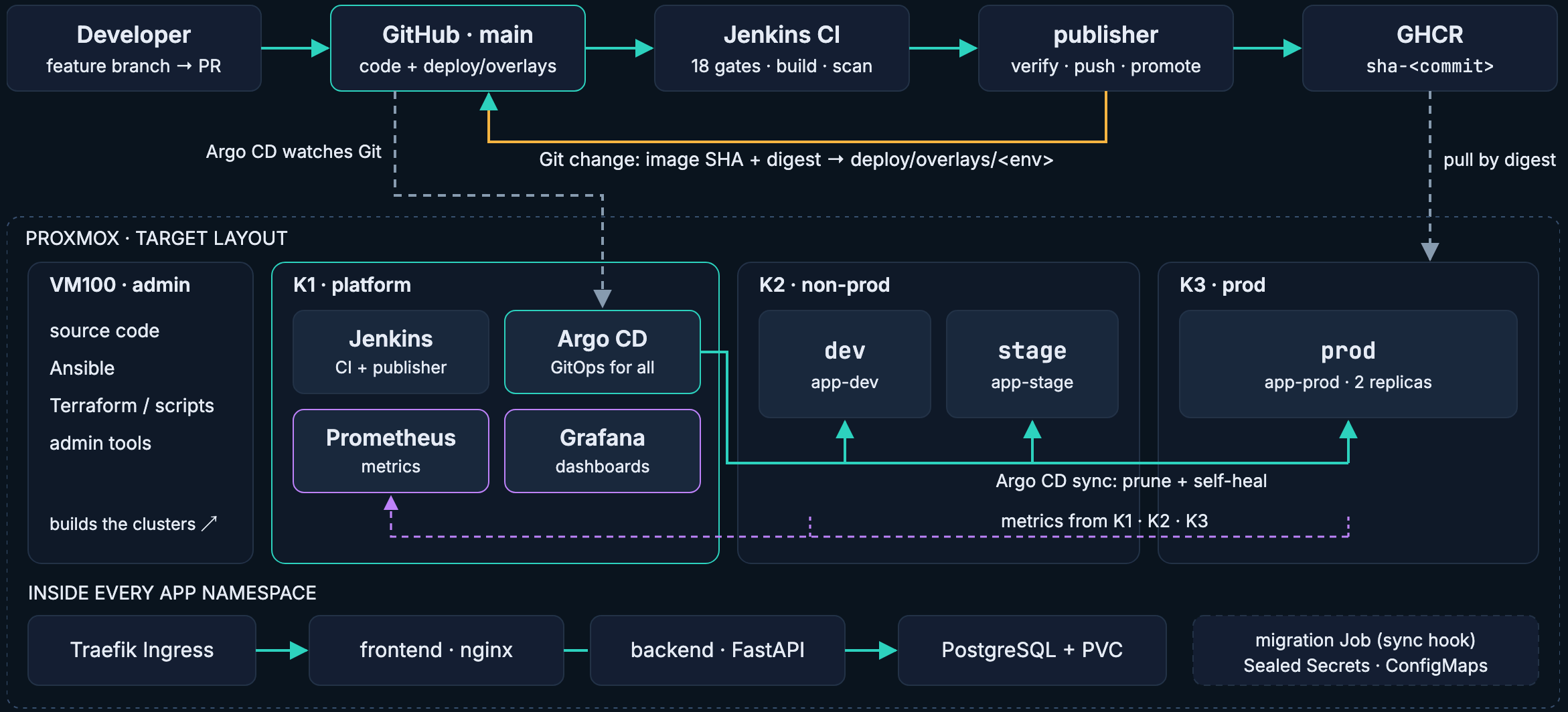
THE APP — DELIBERATELY THE STOCK TEMPLATE

- **Backend:** FastAPI · SQLModel · PostgreSQL 16 · JWT auth · Alembic migrations
- **Frontend:** React · TypeScript · Vite · Chakra UI, served by nginx
- **Edge:** Traefik — `/api` → backend, `/` → frontend

Based on the open-source Full Stack FastAPI Template — the focus is the delivery platform, not the app code.



02 Architecture · target



03 Infrastructure & IaC: Proxmox + k3s

3 + 1
k3s clusters (platform, non-prod, prod) + admin VM

4 / 8 / 100
vCPU · GB RAM · GB HDD per node

HA + 1 VIP
3 servers, embedded etcd; kube-vip API address

8 cores
physical CPU, the real bottleneck

IAC · RUN FROM VM100

1. **VM template:** Debian 13 cloud-init image `SCRIPT`
2. **Node VMs** cloned from the template `SCRIPT` `→ TERRAFORM`
3. **Ansible** `site.yml`: prepare nodes → first server → kube-vip → join servers → kubeconfig `DONE`
4. **Jenkins:** config as code (JCasC, Job DSL, pinned plugins + tools) `DONE`
5. **Apps:** Argo CD ApplicationSet re-creates every env from Git `DONE`

k3s version pinned · secrets in Ansible Vault · kube-vip manifest vendored locally

TARGET LAYOUT · IN PROGRESS

- **VM100 · admin:** code, Ansible, Terraform, tools
- **K1 · platform:** Jenkins, Argo CD, Prometheus, Grafana
- **K2 · non-prod:** `dev` + `stage`
- **K3 · prod:** `prod`
- Today: Jenkins in Docker; apps tested on single-node **minikube**

04 Docker & registry

Backend

Multi-stage: uv build → Python 3.13 Alpine runtime. **Non-root** (UID 10001), healthcheck, no build or test tools inside.

Frontend

Multi-stage: Node 24 build → **unprivileged nginx** (UID 101), healthcheck, only static files.

- Base images **pinned by digest**; locked dependencies (`uv.lock`, `npm ci`)
- Registry: **GHCR** — `fastapi-backend`, `fastapi-frontend`
- Tag = `sha-<full commit SHA>`, **write-once**; deployed by **digest**
- Every image: non-root health smoke + Trivy scan before it may be published



Screenshot: GHCR package page with `sha-…` tags

05 Environments: dev · stage · prod

	DEV	STAGE	PROD
Runs	any selected branch	latest green `main`	release `vX.Y.Z`
Updated by	manual Jenkins run `MODE=dev`	automatically after each merge	merging the release PR
Gate	green CI + Jenkins approval	full CI on `main`	tag + Jenkins approval + PR review
Replicas (be / fe)	1 / 1	1 / 1	2 / 2
Uvicorn workers	1	1	4
Host	`dev.liora.aprt.info` · HTTP	`stage.liora.aprt.info` · TLS	`liora.aprt.info` · TLS
`/docs` exposed	yes	no	no
Target cluster	K2 · non-prod	K2 · non-prod	K3 · prod

How: Kustomize `deploy/base` + one overlay per env → namespaces `app-dev` / `app-stage` / `app-prod`. Config via ConfigMap patches, secrets via Sealed Secrets.

TBD: first prod deployment · switch the `*.example.com` placeholders in the overlays to the `liora.aprt.info` hosts + DNS.

06 Tests

55
backend: 2 unit + 53 API / integration

89%
backend coverage

41
Playwright E2E tests

18
required CI gates

- **Where:** Jenkins CI agent, every build — throwaway PostgreSQL + Mailpit containers, no host ports
- Also: mypy, Ruff, Biome, TS + Vite build, CI-script tests (19)
- **Any failed gate → no image → nothing to publish**

> **TBD · tests on the deployed app:** smoke test as an Argo CD **PostSync hook** (health, OpenAPI, frontend via Traefik) on every sync to K2 / K3



Screenshot: Jenkins pipeline

18 green stages + test results

07 Kubernetes manifests

Deployments
backend, frontend, postgres, adminer

Services + Ingress
ClusterIP, Traefik

ConfigMaps + Secrets
sealed, per namespace

PVC + Job
DB storage, migrations



ARGO CD SYNC ORDER

1. wave `-2`: config, storage, PostgreSQL
2. wave `0`: DB migration Job (sync hook) — failure blocks rollout
3. wave `1`: backend + frontend rollout

- Probes, CPU + memory limits, no service-account tokens
- PostgreSQL `Recreate`; PVC protected from Argo prune

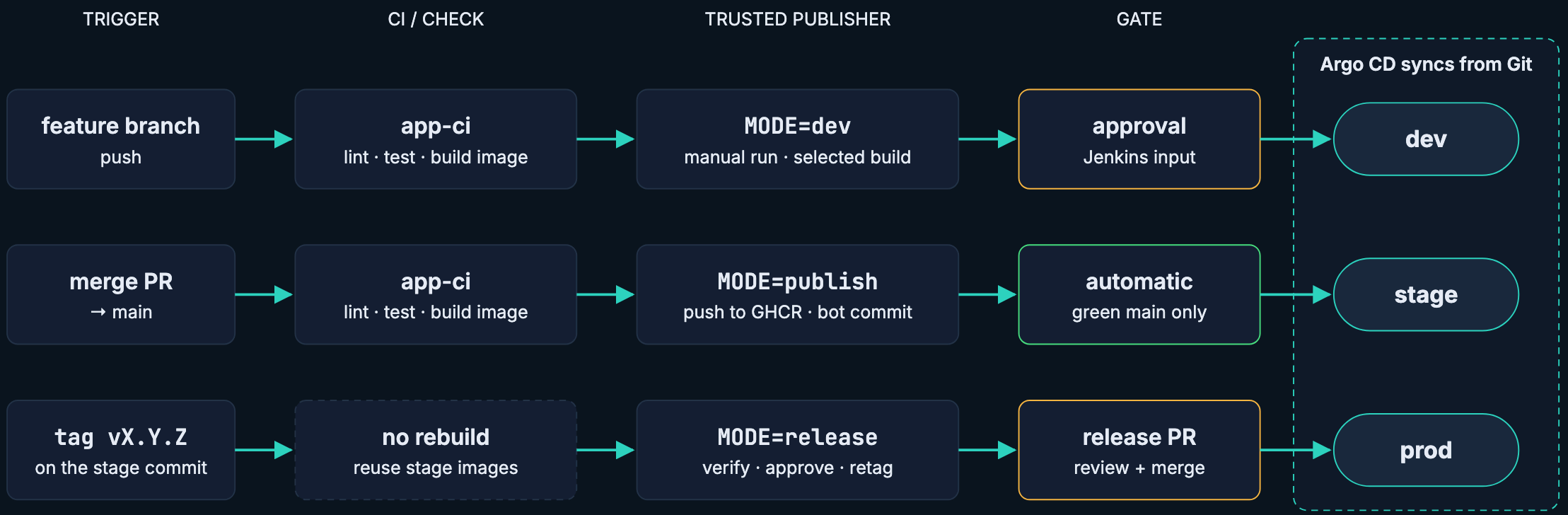
TBD · HELM CHART FOR THE APP (BONUS) · HPA

08 CI/CD — the flow

GitHub Flow model — one `main`, always deployable; short-lived branches + PRs

GitOps — Jenkins never touches the app cluster; it commits which image runs where

Build once, deploy many — same image dev → stage → prod; only config differs



PHASE 1 · DONE CI → 18 gates → verified images → GHCR

PHASE 2 · NEXT Git promotion → Argo CD → K2 / K3

09 CI/CD — Jenkins jobs, release & rollback

`app-ci` · CI agent

18 gates: secret scan → lint → unit / API / E2E → dependency audits → Docker build → non-root smoke → Trivy → attest. **No credentials.**

`trusted/publish` · publisher agent

Separate agent. Checks CI = SUCCESS, Git SHA, all gates and archive hashes → pushes `sha-` tags to GHCR. Refuses a changed tag.

PHASE 2 · RELEASE `V1.2.0`

1. Tagged commit must be on `main`
2. `sha-<commit>` images exist in GHCR and match stage history
3. Jenkins approval: "this ran healthy in stage"
4. Retag existing image as `v1.2.0` — **no rebuild**
5. `release/v1.2.0` → PR to prod overlay → **merge = approval**

ROLLBACK = GIT REVERT

- Every deployment is a commit → revert it, Argo CD syncs in minutes
- No rebuild: old digests stay in GHCR
- DB migrations aren't auto-reversible → check schema

VERIFIED full Phase 1 CI green (18 gates); earlier on minikube: CI → GHCR → Argo synced stage

NOT YET Phase 1 run from `main` + GHCR push · promotion · prod

10 CI/CD — design decisions

WE CHOSE	INSTEAD OF	WHY
GitHub Flow branching model	GitFlow / a `dev` branch	One service, continuous delivery: no need for develop/release/hotfix branches. Versions = tags. A dev environment exists — a dev branch isn't needed.
Artifact promotion	environment branches	Promoting the same image is safer than merging code between branches that drift apart
Argo CD pull model	`kubectl apply` from Jenkins	No cluster credentials in CI; Git is the audit log; drift is auto-corrected
Two Jenkins jobs	one pipeline with `when main`	A branch author can edit the Jenkinsfile — only a separate job is a real credentials boundary
Manifests in the app repo	separate config repo	Self-contained for one service; with more services → config repo + Argo CD Image Updater
Shared dev	per-PR environments	Cheap on our 8 cores; later an ApplicationSet PR generator for previews

GitHub Flow is a branching model, not a GitHub product: our CI/CD tooling is **Jenkins + Argo CD**, not GitHub Actions.

11 Security (DevSecOps)

Image scanning

Trivy on every image: HIGH / CRITICAL or a secret **blocks** the build. Jenkins images: 0 High / Critical.

Dependencies

pip-audit + **npm audit** as blocking gates, **Dependabot** for updates. npm: **62 → 0** vulnerabilities.

Secrets

Sealed Secrets in Git; **Gitleaks** scans code + full Git history; CI test secrets generated per run.

Secure pipeline

Only the publisher holds the GHCR token; CI agent has none. Builds in isolated Docker-in-Docker over TLS, no host socket.

Hardened runtime

Non-root images, digest-pinned bases, healthchecks; TLS on stage / prod ingress; Adminer not exposed.

RBAC (bonus)

Argo AppProject whitelist: no Secrets, Namespaces or cluster resources; pods without SA tokens.

Known limits: Docker-in-Docker stays privileged (isolated CI host, no credentials) · untrusted fork PRs would need throwaway workers · cert-manager for TLS is next

12 Monitoring & disaster recovery

MONITORING · IN PROGRESS



Screenshot: Grafana dashboard

Prometheus + Grafana on K1, metrics from K1 · K2 · K3

Today: k8s probes, metrics-server, Argo CD sync/health status, Jenkins test reports

RECOVERY · DONE

Container fails	probes → Deployment restarts it
Node fails	HA k3s: etcd quorum + kube-vip survive 1 of 3 nodes; pods rescheduled
Manual drift	Argo CD self-heal restores Git state
Bad release	`git revert` → Argo syncs the previous digests
Lost cluster	rebuild with Ansible → Argo CD re-creates apps from Git

NOT DONE YET

- **DB backups:** CronJob `pg_dump` → off-cluster storage + restore test (or Velero)
- Postgres PVC is node-local → losing that node loses data until backups exist

13 Status & roadmap

1 · Docker	DONE multi-stage, non-root
2 · Environments	3 CONFIGURED · PROD PENDING
3 · Tests	DONE unit + API + E2E
4 · Kubernetes	DONE Helm chart: TBD
5 · CI/CD	PHASE 1 DONE · PHASE 2 NEXT
6 · IaC	ANSIBLE + JCASC ✓ · TERRAFORM
7 · Monitoring	IN PROGRESS
8 · Security	6 AREAS, BLOCKING GATES
9 · Disaster recovery	NO DB BACKUPS
10 · Documentation	README + DIAGRAM

ROADMAP TO THE FINAL DEFENSE

1. Phase 2: Git promotion + Argo CD → K2 / K3
2. K1 platform: Jenkins, Argo CD, Prometheus, Grafana
3. Grafana dashboards + alerts
4. Terraform for the Proxmox VMs
5. PostgreSQL backups + restore test
6. Smoke tests after each Argo CD sync
7. cert-manager for TLS
8. README rewrite + architecture diagram

14 Challenges & lessons learned

kube-vip.io blocked by our host

The install fetched the kube-vip RBAC manifest from `kube-vip.io` — blocked from our server. We vendored `rbac.yaml` into the scripts, then into Ansible.
Lesson: no external URLs at install time — pin and vendor.

30+ minute builds

BuildKit saturated the shared 8 cores; every k8s CI pod started with an empty cache. Now: permanent Docker agents with a persistent build cache.
Lesson: caching matters more than more clusters.

Jenkins disk full

Builds failed with `ENOSPC` on minikube → investigation report + runbook.
Lesson: a Running pod ≠ a healthy pipeline.

Security clean-up

npm 62 → 0 vulns; Jenkins agent 47 High + 1 Critical → 0 by rebuilding tools from pinned sources. Kaniko → BuildKit; k8s agents → Docker agents.
Lesson: scanners must block, not warn.

Thank you — questions?

Vitalii TYTSKYI · Dionis FRANGU · Tatiana ZAINULLINA

`github.com/fastapi-traefik-devops`